

Doubly Linked List Operations

Create a **Doubly Linked List** and implement the following operations:

1. Traversal

- Traverse the doubly linked list **forward** and display all the elements.
- Traverse the doubly linked list **backward** and display all the elements.

2. Insertion

- Insert a new node at the **beginning** of the doubly linked list.
- Insert a new node at the **end** of the doubly linked list.
- Insert a new node at a **specific position** in the doubly linked list.

3. Deletion

- Delete the node from the **beginning** of the doubly linked list.
- Delete the node from the **end** of the doubly linked list.
- Delete the node from a **specific position** in the doubly linked list.

4. Searching

- Search for a given element in the doubly linked list and determine whether it is present.

5. Updation

- Search for a given element and update its value with a new value.

6. Reversal

- Reverse the entire doubly linked list by appropriately updating both the `next` and `prev` pointers.

Creation

```
public class DoublyLinkedList {
    int val;
    DoublyLinkedList next;
    DoublyLinkedList prev;
    static DoublyLinkedList head;

    DoublyLinkedList(){

    }

    DoublyLinkedList(int val){
        this.val = val;
    }

    DoublyLinkedList(int val, DoublyLinkedList next, DoublyLinkedList prev){
        this.val = val;
        this.next = next;
        this.prev = prev;
    }
}
```

1. Traversal

1a. Traversal Forward

```
private static void TraversalForward(DoublyLinkedList head){
    DoublyLinkedList curr = head;

    while (curr != null){
        System.out.print(curr.val + "↔");
        curr = curr.next;
    }
    System.out.println("null");
}
```

1b. Traversal Backward

```
private static void TraversalBackward(DoublyLinkedList head) {
    if (head == null) {
        System.out.println("null");
        return;
    }

    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    while (curr != null) {
        System.out.print(curr.val + "↔");
        curr = curr.prev;
    }

    System.out.println("null");
}
```

2. Insertion

2a. Insert at the Start

```
private static DoublyLinkedList InsertAtStart(DoublyLinkedList node) {
    node.next = head;

    if (head != null) {
        head.prev = node;
    }

    head = node;

    return head;
}
```

2b. Insert at the end

```
private static DoublyLinkedList InsertAtEnd(DoublyLinkedList node) {
    if (head == null) {
        head = node;
        return head;
    }
```

```

    }

    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    curr.next = node;
    node.prev = curr;

    return head;
}

```

2c. Insert at the specific position

```

private static DoublyLinkedList InsertAtPosition(DoublyLinkedList node, int position) {
    if (position < 1) {
        System.out.println("Position Not Found");
        return head;
    }

    if (position == 1) {
        node.next = head;

        if (head != null) {
            head.prev = node;
        }

        head = node;
        return head;
    }

    DoublyLinkedList curr = head;
    int counter = 1;

    while (curr != null && counter < position - 1) {
        curr = curr.next;
        counter++;
    }

    if (curr != null) {
        node.next = curr.next;
        node.prev = curr;

        if (curr.next != null) {
            curr.next.prev = node;
        }

        curr.next = node;
    } else {
        System.out.println("Position Not Found");
    }

    return head;
}

```

3. Deletion

3a. Delete at Start

```
private static DoublyLinkedList DeleteAtStart() {
    if (head == null) return null;

    head = head.next;

    if (head != null) {
        head.prev = null;
    }

    return head;
}
```

3b. Delete at End

```
private static DoublyLinkedList DeleteAtEnd() {
    if (head == null) {
        return null;
    }

    if (head.next == null) {
        head = null;
        return head;
    }

    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    curr.prev.next = null;

    return head;
}
```

3c. Delete at the specific position

```
private static DoublyLinkedList DeleteAtPosition(int position) {
    if (head == null) return null;

    if (position < 1) {
        System.out.println("Position Not Found");
        return head;
    }

    if (position == 1) {
        head = head.next;

        if (head != null) {
            head.prev = null;
        }
    }
}
```

```

        return head;
    }

    DoublyLinkedList curr = head;
    int counter = 1;

    while (curr != null && counter < position - 1) {
        curr = curr.next;
        counter++;
    }

    if (curr != null && curr.next != null) {
        curr.next.next.prev = curr;
        curr.next = curr.next.next;
    } else {
        System.out.println("Position Not Found");
    }

    return head;
}

```

4. Searching

```

private static boolean CheckIfPresent(int value){
    DoublyLinkedList curr = head;

    while (curr != null){
        if(curr.val == value){
            return true;
        }
        curr = curr.next;
    }

    return false;
}

```

5. Updation

```

private static DoublyLinkedList UpdateElement(int oldValue , int newValue){
    DoublyLinkedList curr = head;

    while (curr != null){
        if(curr.val == oldValue){
            curr.val = newValue;
            break;
        }

        curr = curr.next;
    }

    return head;
}

```

6. Reverse

```
private static DoublyLinkedList ReverseList(DoublyLinkedList head) {
    DoublyLinkedList curr = head;
    DoublyLinkedList newHead = null;

    while (curr != null) {
        DoublyLinkedList temp = curr.next;

        curr.next = curr.prev;
        curr.prev = temp;

        newHead = curr;
        curr = temp;
    }

    return newHead;
}
```

Main Code

Code

```
public static void main(String[] args){
    DoublyLinkedList one = new DoublyLinkedList(1);
    DoublyLinkedList two = new DoublyLinkedList(2);
    DoublyLinkedList three = new DoublyLinkedList(3);
    DoublyLinkedList four = new DoublyLinkedList(4);

    one.next = two;
    two.prev = one;

    two.next = three;
    three.prev = two;

    three.next = four;
    four.prev = three;

    head = one;

    System.out.println("Printing the Doubly Linked while traversing forward");
    TraversalForward(head);

    System.out.println("Printing the Doubly Linked while traversing backward");
    TraversalBackward(head);

    DoublyLinkedList zero = new DoublyLinkedList(0);

    System.out.println("Adding node 0 at the start");
    InsertAtStart(zero);
    TraversalForward(head);

    DoublyLinkedList five = new DoublyLinkedList(5);

    System.out.println("Adding node 5 at the end");
    InsertAtEnd(five);
}
```

```

        TraversalForward(head);

        DoublyLinkedList six = new DoublyLinkedList(6);

        System.out.println("Inserting at node 6 at position 2");
        InsertAtPosition(six,2);
        TraversalForward(head);

        System.out.println("Deleting at the start");
        DeleteAtStart();
        TraversalForward(head);

        System.out.println("Deleting at the end");
        DeleteAtEnd();
        TraversalForward(head);

        System.out.println("Deleting at position 2");
        DeleteAtPosition(2);
        TraversalForward(head);

        System.out.println("Checking if 6 is present or not :- "+CheckIfPresent(6));

        System.out.println("Checking if 5 is present or :- "+ CheckIfPresent(5));

        System.out.println("Updating node 3 value with 5");
        UpdateElement(3,5);
        TraversalForward(head);

        System.out.println("Reversing the List");
        head = ReverseList(head);
        TraversalForward(head);

    }

```

Output

```

Printing the Doubly Linked while traversing forward
1⇄2⇄3⇄4⇄null
Printing the Doubly Linked while traversing backward
4⇄3⇄2⇄1⇄null
Adding node 0 at the start
0⇄1⇄2⇄3⇄4⇄null
Adding node 5 at the end
0⇄1⇄2⇄3⇄4⇄5⇄null
Inserting at node 6 at position 2
0⇄6⇄1⇄2⇄3⇄4⇄5⇄null
Deleting at the start
6⇄1⇄2⇄3⇄4⇄5⇄null
Deleting at the end
6⇄1⇄2⇄3⇄4⇄null
Deleting at position 2
6⇄2⇄3⇄4⇄null
Checking if 6 is present or not :- true
Checking if 5 is present or :- false
Updating node 3 value with 5
6⇄2⇄5⇄4⇄null

```

Reversing the List

4↔5↔2↔6↔null